

Splitit

ID: a8051faf-9820-4c51-96ae-5dc9d9dc855c **State:** COMPLETE **Exported:** 2026-06-09 19:46 UTC
Type: hybrid

Executive summary

This architecture illustrates a decoupled, microservices-oriented approach to order processing, where a single "Create Order" request from a **Client App** is decomposed into multiple internal API calls.

The **Web Tier** receives the initial request and forwards it as a `POST /orders` to the **Order API**. Acting as an orchestrator, the Order API manages a sequence of synchronous calls to internal services: validating the customer, reserving stock, and charging the order.

To ensure high performance and reliability, the system employs an asynchronous pattern for non-critical paths. Once the core order is validated, the Order API saves the data to the **Order Database** and publishes an event to a **Message Queue**. This triggers a downstream **Notification Service** and interacts with a **Third-Party External Payment Gateway**. Finally, an **Order Confirmation** is sent back to the client, demonstrating a system designed for scalability and improved transaction management speed.

Assets & entities

TYPE	NAME	CRITICALITY	DESCRIPTION
Entity	Client App	High	The front-end interface used by end-users to initiate "Create Order" requests.
Entity	Web Tier	High	The entry point/gateway that receives external traffic and forwards requests to internal APIs.

Data flows

SOURCE	TARGET	DESCRIPTION
Client App	Web Tier	"Initial 'Create Order' request from the end-user."
Web Tier	Order API	"Forwarded POST /orders request."
Order API	Customer Validation Service	"Synchronous customer validation request."
Order API	Stock Service	"Synchronous stock reservation request."
Order API		"Synchronous charge request."

SOURCE	TARGET	DESCRIPTION
	Payment Processing Logic (Internal)	
Order API	Order Database	"Persistence of order data."
Order API	Message Queue	"Order created event publication."
Message Queue	Notification Service	"Asynchronous consumption of order event."
Order API (or Notification Service/Worker)	Third-Party External Payment Gateway	"External payment gateway interaction."
Order API	Client App	"Final Order Confirmation response."

Threat actors

CATEGORY	DESCRIPTION	EXAMPLES
External Adversaries	Malicious actors attempting to exploit public-facing endpoints or intercept data.	Script kiddies, Hacktivists, Organized Cybercrime groups.
Insider Threats	Employees or contractors with authorized access to internal systems/databases.	Malicious admins, disgruntled developers, compromised employee credentials.
Third-Party Risks	Vulnerabilities or breaches originating from the external payment gateway or message queue provider.	Supply chain attacks, API key theft from the gateway.
Automated Bots	Non-human actors attempting to brute force orders or scrape data.	Credential stuffing bots, inventory hoarding bots.

Threat catalog (STRIDE)

TOTAL THREATS

12

Unauthorized Order Creation via Web Tier

SPOOFING

MEDIUM

External Adversaries, by bypassing client-side validation, can spoof a legitimate user's identity to send a POST /orders request which leads to unauthorized order placement, negatively impacting the Client App.

Target: Order API · **Source:** External Adversaries

Vector: Network / API Request

Impact: High

Mitigations: Implement JWT or OAuth2 authentication for all requests; Enforce TLS for all data in transit; Validate API keys or session tokens at the Web Tier and Order API

Order Data Modification in Transit

TAMPERING

LOW

External Adversaries, by intercepting the forwarded POST /orders request, can modify order quantities or pricing which leads to financial loss and inventory discrepancies, negatively impacting the Order Database.

Target: Order API · **Source:** External Adversaries

Vector: Network / MitM

Impact: High

Mitigations: Enforce HTTPS/TLS 1.3 for all internal and external communications; Implement request signing or HMAC to ensure integrity; Use payload validation schemas at the Order API

Inventory Exhaustion via Automated Bots

DENIAL OF SERVICE

HIGH

Automated Bots, by sending a high volume of synchronous stock reservation requests, can exhaust available inventory which leads to service unavailability for legitimate users, negatively impacting the Stock Reservation Service.

Target: Stock Reservation Service · **Source:** Automated Bots

Vector: Network / Flooding

Impact: Medium

Mitigations: Implement Rate Limiting and Throttling; Use CAPTCHA or WAF rules to identify bot patterns; Implement circuit breakers for downstream service calls

Payment Information Leakage to Third-Party

INFORMATION DISCLOSURE

MEDIUM

Third-Party Risks, by compromising the external payment gateway or intercepting the charge request, can access sensitive customer credit card data which leads to regulatory non-compliance (PCI-DSS) and data breach, negatively impacting the Client App.

Target: Third-Party External Payment Gateway · **Source:** Third-Party Risks

Vector: Third-Party Integration

Impact: Critical

Mitigations: Use tokenization (e.g., Stripe/Braintree) so raw data never hits your server; Encrypt all PII and sensitive data at rest and in transit; Conduct regular security audits of third-party providers

Message Queue Poisoning

TAMPERING

LOW

Insider Threats, by gaining access to the Message Queue, can inject malicious messages which leads to the Notification Service sending fraudulent alerts or executing unintended actions, negatively impacting the Notification Service.

Target: Message Queue · **Source:** Insider Threats

Vector: Internal Network / System Access

Impact: High

Mitigations: Implement strict Access Control Lists (ACLs) for the Message Queue; Encrypt messages in the queue; Use message signing to verify the source of the event

Order Denial via Database Overflow

DENIAL OF SERVICE

MEDIUM

External Adversaries, by exploiting the Order API to create a massive volume of valid-looking orders, can overwhelm the Order Database which leads to system-wide crash and inability to process orders, negatively impacting the Order Database.

Target: Order Database · **Source:** External Adversaries

Vector: Application Logic

Impact: High

Mitigations: Implement Database connection pooling; Set quotas on the number of orders per user/account; Use an asynchronous buffer (Message Queue) for database writes if possible

Client-Side Request Forgery / Identity Spoofing

SPOOFING

MEDIUM

External Adversaries can spoof a legitimate user's identity in the 'Initial Create Order request' by stealing session tokens, which leads to unauthorized order placement, negatively impacting Client App.

Target: Client App · **Source:** External Adversaries

Vector: Network

Impact: High

Mitigations: Implement Multi-Factor Authentication (MFA); Use Secure, HttpOnly, SameSite cookies; Implement OAuth2/OIDC with short-lived JWTs

Web Tier Infrastructure Exhaustion

DENIAL OF SERVICE

HIGH

Automated Bots can flood the 'Forwarded POST /orders request' flow with high-volume traffic, which leads to service unavailability for legitimate users, negatively impacting Web Tier.

Target: Web Tier · **Source:** Automated Bots

Vector: Network

Impact: High

Mitigations: Implement Rate Limiting and Throttling; Deploy a Web Application Firewall (WAF); Use Content Delivery Network (CDN) for edge protection

Order Request Parameter Tampering

TAMPERING

MEDIUM

External Adversaries can modify the price or quantity fields in the 'Forwarded POST /orders request' before it reaches the Order API, which leads to financial loss and inventory discrepancies, negatively impacting Web Tier.

Target: Web Tier · **Source:** External Adversaries

Vector: Application

Impact: High

Mitigations: Perform server-side validation of all prices/quantities; Use HMAC or digital signatures for request integrity; Implement strict input schema validation

Message Queue Injection / Poisoning

TAMPERING

LOW

Insider Threats can inject malicious payloads into the 'Message Queue' after the Order API publishes an event, which leads to unauthorized actions in the Notification Service or downstream systems, negatively impacting Order Database.

Target: Order Database · **Source:** Insider Threats

Vector: Internal Network

Impact: High

Mitigations: Implement Message Authentication Codes (MAC); Strict Schema validation for all queue messages; Principle of Least Privilege for queue access

Payment Gateway Information Leakage

INFORMATION DISCLOSURE LOW

Third-Party Risks can intercept or log sensitive payment details during the 'Synchronous charge request' if the communication is not properly encrypted, which leads to PCI-DSS non-compliance and identity theft, negatively impacting Client App.

Target: Client App · **Source:** Third-Party Risks

Vector: Network

Impact: Critical

Mitigations: Ensure TLS 1.3 for all external communications; Use Tokenization for payment details; Implement strict logging redaction for PII/PCI data

Stock Reservation Race Condition

DENIAL OF SERVICE MEDIUM

Automated Bots can exploit the 'Synchronous stock reservation request' by sending concurrent requests for the last remaining item, which leads to over-selling or system locking, negatively impacting Web Tier.

Target: Web Tier · **Source:** Automated Bots

Vector: Application

Impact: Medium

Mitigations: Implement Distributed Locking (e.g., Redis Lock); Use Atomic Database Transactions; Implement Idempotency Keys for all orders

Colophon

Maestro'D ThreatModeling
Copyright 2026 Maître D and contributors
Inspired by AWS Threat Designer (<https://github.com/awslabs/threat-designer>), licensed under the Apache License 2.0.
Envisioned by Dirk Verstraete (Maître D).
Designed and architected with Lead Architect Maestro Data (Cursor agent).
Local model — weapon of choice: Gemma 4 12B (coding and threat modeling).
This project is not affiliated with or endorsed by Amazon Web Services, Inc.
Report generated 2026-06-09 19:46 UTC · Model a8051faf-9820-4c51-96ae-5dc9d9dc855c